1.1 BDD from 50,000 feet

So what does BDD bring to the table? Here's a (slightly oversimplified) perspective. Let's say Chris's company needs a new module for its accounting software. When Chris wants to add a new feature, the process goes something like this (see figure 1.1):

1. Chris tells a business analyst how he would like the feature to work.
2. The business analyst translates Chris's requests into a set of requirements for the developers, describing what the software should do. These requirements are written in English and stored in a Microsoft Word document.
3. The developer translates the requirements into code and unit tests—written in Java, C#, or some other programming language—in order to implement the new feature.
4. The tester translates the requirements in the Word document into test cases, and uses them to verify that the new feature meets the requirements.
5. Documentation engineers then translate the working software and code back into plain English technical and functional documentation.

There are many opportunities for information to get lost in translation, be misunderstood, or just be ignored. Chances are that the new module itself may not do exactly what was required and that the documentation won't reflect the initial requirements that Chris gave the analyst.

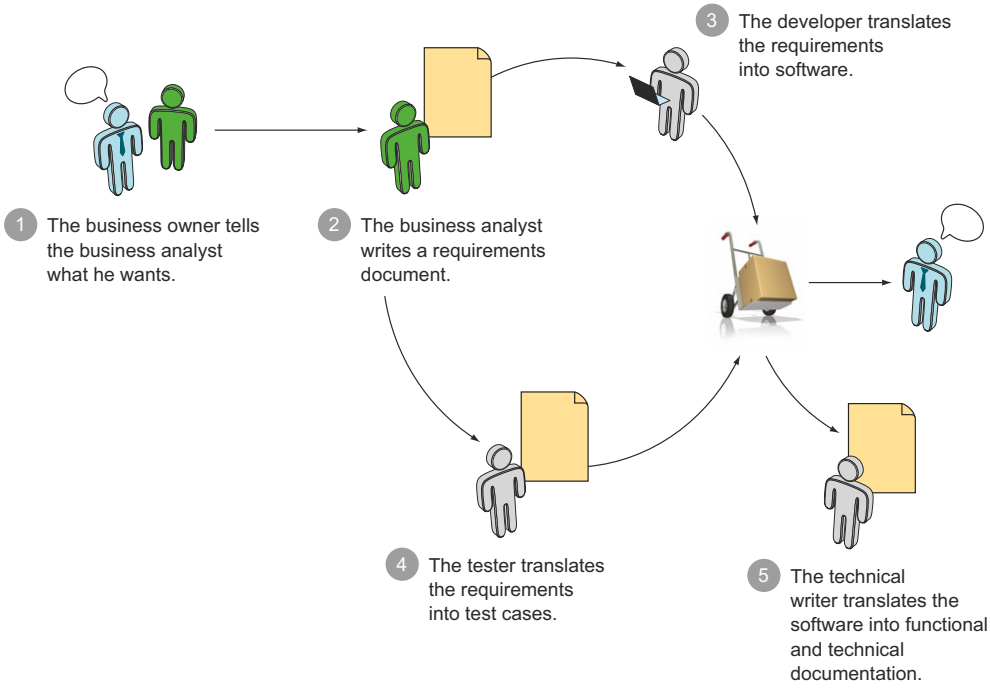


Figure 1.1 The traditional development process provides many opportunities for misunderstandings and miscommunication.